AXI-FDI Bridge — Project Specification

Revision 1.0 — DRAFT FOR REVIEW

IP Name: `AXFDI_BRIDGE` **Standard Alignment:** UCIe 2.0 — Streaming Protocol (Flit Format 6) **Date:** 2026-04-02 **Status:** Architecture Phase — Pending Team Review

---

Table of Contents

AXI-FDI Bridge — Project Specification
- Revision 1.0 — DRAFT FOR REVIEW
- Table of Contents
- 1. Introduction & Scope
  - 1.1 Purpose
  - 1.2 Scope
  - 1.3 Out of Scope
- 2. Terminology & Abbreviations
- 3. Architecture Overview
  - 3.1 High-Level Block Diagram
  - 3.2 Symmetric Operation
  - 3.3 Transaction Flow Examples
- 4. Interface Specifications
  - 4.1 AXI4 Full Subordinate Interfaces — AXFSUB[0] and AXFSUB[1]
  - 4.2 AXI4 Full Manager Interfaces — AXFMGR[0] and AXFMGR[1]
  - 4.3 FDI Mainband Interface
  - 4.4 APB Sideband / CSR Interface
- 5. UCIe 2.0 Streaming Flit Format
  - 5.1 Format 6 — Streaming Identification and CRC Reservation
  - 5.2 Header Flit Definition (256 bytes = 2048 bits total)
  - 5.3 Data Flit Definition (256 bytes = 2048 bits total)
  - 5.4 Opcode Definitions
  - 5.5 Write Payload Stream Ordering (WSTRB and Data)
  - 5.6 Virtual Channel Assignments
- 6. TX Data Path — AXI → FDI
  - 6.1 TX Request Buffers
  - 6.2 TX Port Merge and Packetizer Arbitration
    - 6.2.1 Port Merge — Simple Alternation

6.2.2 Transaction-Type Arbitration — WRR within Packetizer
6.3 Packetizer
6.4 TX CDC — AXI Domain Boundary (500 MHz → 625 MHz)
7. RX Data Path — FDI → AXI
7.1 RX CDC — AXI Domain Boundary (625 MHz → 500 MHz)
7.2 Depacketizer
7.3 Port Router and OPCODE Dispatch
7.4 RX Data Buffers (per port)
8. Credit Flow Management
8.1 Credit Model
8.2 Credit Initialization
8.3 Credit Return Mechanism
8.4 Deadlock Avoidance
8.5 Credit Counter CSR Visibility
9. Clock Domain Crossing Strategy
9.1 Clock Domains
9.2 CDC Placement and Structure
9.3 CDC Design Rules
10. CSR Map (APB-accessible)
10.1 Global Control
10.2 Credit Configuration
10.3 Credit Status (Debug)
10.4 Error and Interrupt Status
10.5 Performance Counters (optional, clock-gated)
11. Error Handling
11.1 AXI Protocol Errors
11.2 FDI Protocol Errors
11.3 Error Recovery
12. Reset Architecture
12.1 Reset Domains
12.2 Reset Sequence
13. Performance Model
13.1 Bandwidth Analysis
13.2 Packing Efficiency
13.3 Latency Model (Baseline, unloaded)
14. RTL / DUT Implementation Spec
14.1 Module Hierarchy
14.2 Key RTL Parameters (Localparam / Parameter)
14.3 Coding Guidelines
14.4 Clock/Reset Port Naming Convention
14.5 Lint and CDC Checks

14.6 Functional Coverage Points (RTL perspective)
15. UVM Testbench Spec
15.1 TB Architecture Overview
15.2 VIP Definitions
15.2.1 VIP-1: AXI4 Full Agent (axi4_agent)
15.2.2 VIP-2: FDI Mainband Agent (fdi_agent)
15.2.3 VIP-3: APB Agent (apb_agent)
15.3 Scoreboard (axfdi_scoreboard)
15.4 Protocol Checkers (SVA)
15.5 Test Plan Summary
15.6 Coverage Model
15.7 Simulation Environment
16. Open Items / Assumptions Log

1. Introduction & Scope

1.1 Purpose

The **AXFDI_BRIDGE** is a fully symmetric, bidirectional protocol conversion IP that bridges two AXI4 Full ports on the local SoC die side to a single UCIe 2.0 Flex Die Interface (FDI) mainband link on the D2D interconnect side. The same RTL instance is instantiated on both Die A and Die B, making the design inherently symmetric and simplifying integration and verification.

1.2 Scope

This specification covers:

- AXI4 Full subordinate interface (2 ports, inbound traffic from local SoC masters)
- AXI4 Full manager interface (2 ports, outbound traffic to local SoC fabric for remote-originated requests)
- FDI mainband TX/RX interface per UCIe 2.0, using Streaming Flit Format 6
- Credit-based flow control on the FDI mainband (no UCIe sideband credits)
- UCIe sideband interface via APB (CSR programming + sideband messaging)
- Clock Domain Crossing (CDC) between AXI domain (500 MHz) and FDI domain (625 MHz)
- Transaction packetization / depacketization
- Error detection, reporting, and recovery

1.3 Out of Scope

- UCIe PHY layer (assumed present below the FDI boundary)
- UCIe Adapter logic (assumed present between FDI and PHY)
- Cache coherency protocol handling (no CXL.cache/CXL.mem; purely non-coherent AXI4)
- Retransmission logic (handled by UCIe Adapter/PHY)

---

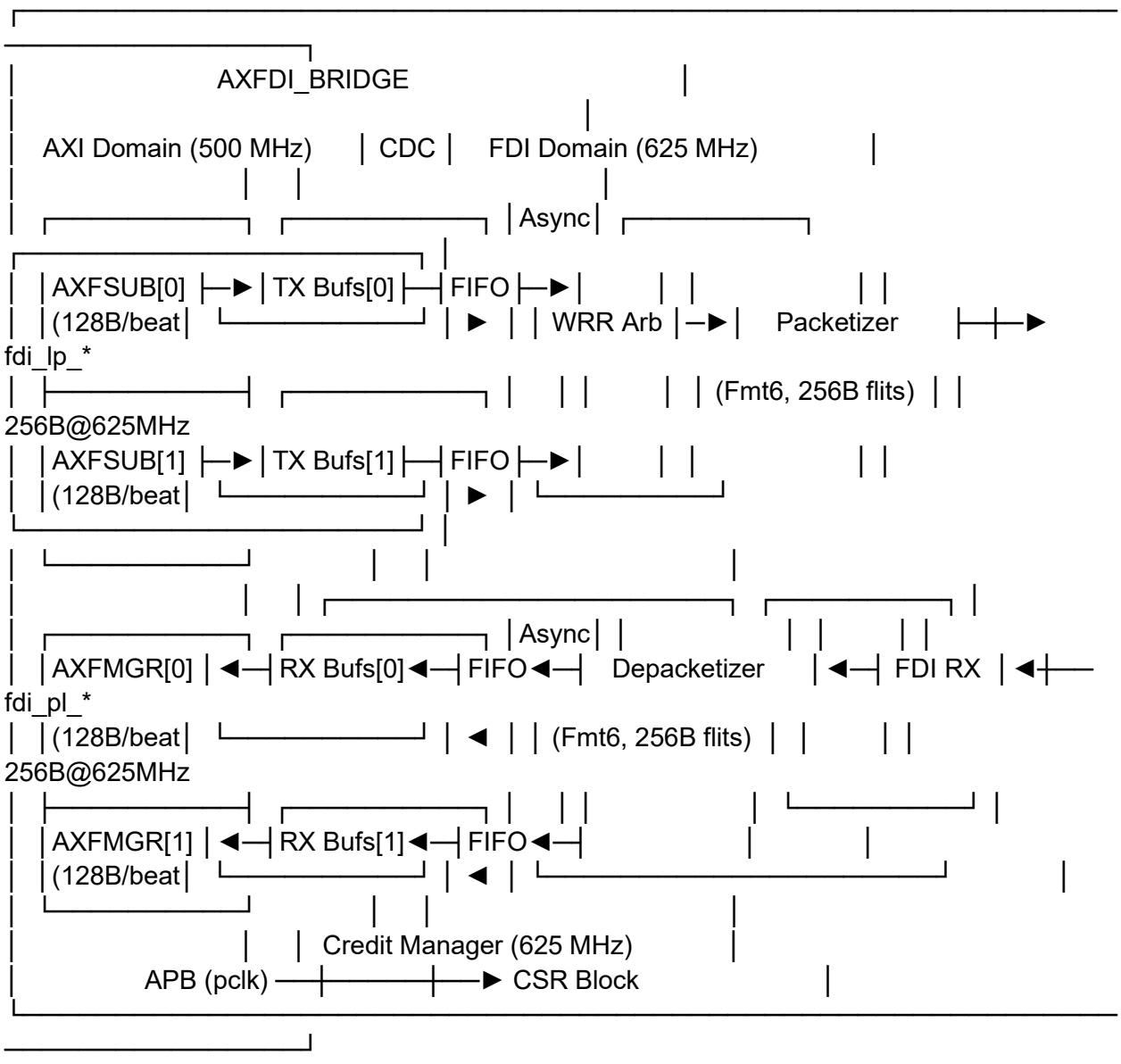
2. Terminology & Abbreviations

Term	Definition
FDI	Flex Die Interface — the boundary between the die protocol logic and the UCIe adapter
UCIe	Universal Chiplet Interconnect Express
AXI	Advanced eXtensible Interface (ARM AMBA AXI4)
LP	Local Protocol — signals originating from the die toward the UCIe adapter (FDI LP direction)
PL	Protocol Layer — signals originating from the UCIe adapter toward the die (FDI PL direction)
Flit	Flow Control Unit — the atomic transfer unit on the FDI mainband
VCN	Virtual Channel Number — identifies a credit/ordering class within the FDI stream
AXFSUB	AXI4 Full Subordinate port (accepts requests from local SoC masters)
AXFMGR	AXI4 Full Manager port (issues requests into local SoC fabric)
WRR	Weighted Round Robin arbitration
CDC	Clock Domain Crossing
CSR	Configuration and Status Register
APB	Advanced Peripheral Bus (for CSR access)
TX	Transmit — local die to remote die direction

RX	Receive — remote die to local die direction
D2D	Die-to-Die
NOP	No Operation (idle flit)
OoO	Out of Order

3. Architecture Overview

3.1 High-Level Block Diagram



Note: All CDC async FIFOs are located at the AXI domain boundary.
Arbiter, Packetizer, Depacketizer, and Credit Manager run at 625 MHz (FDI domain).
AXI protocol handlers and per-port buffers run at 500 MHz (AXI domain).

3.2 Symmetric Operation

The **same RTL** is instantiated on both dies. The FDI link is point-to-point: Die A's LP signals connect to Die B's PL signals through the UCIe adapter+PHY, and vice versa.

- Die A AXFSUB[0/1] → Die A TX → FDI → Die B RX → Die B AXFMGR[0/1]
- Die B AXFSUB[0/1] → Die B TX → FDI → Die A RX → Die A AXFMGR[0/1]

Port binding: The `port_id` field in the flit header (1 bit) identifies which AXFMGR port to use on the receiving die. Port 0 transactions from Die A's AXFSUB[0] are delivered to Die B's AXFMGR[0], and similarly for Port 1. This creates two independent logical channels across the link.

3.3 Transaction Flow Examples

Write (Die A AXFSUB[0] → Die B AXFMGR[0]):

Die A: AW+W [500MHz] → TX Bufs → CDC FIFO → WRR Arb → Packetize [625MHz] → FDI TX
Die B: FDI RX → Depacketize [625MHz] → CDC FIFO → RX Bufs [500MHz] → AXFMGR[0] AW+W
Die B: AXFMGR[0] B-channel [500MHz] → TX Bufs → CDC FIFO → Packetize (WRITE_RESP) [625MHz] → FDI TX
Die A: FDI RX → Depacketize [625MHz] → CDC FIFO → RX Bufs [500MHz] → AXFSUB[0] B-channel

Read (Die A AXFSUB[1] → Die B AXFMGR[1]):

Die A: AR [500MHz] → TX Bufs → CDC FIFO → WRR Arb → Packetize (READ_REQ) [625MHz] → FDI TX
Die B: FDI RX → Depacketize [625MHz] → CDC FIFO → RX Bufs [500MHz] → AXFMGR[1] AR
Die B: AXFMGR[1] R-channel [500MHz] → TX Bufs → CDC FIFO → Packetize (READ_RESP) [625MHz] → FDI TX
Die A: FDI RX → Depacketize [625MHz] → CDC FIFO → RX Bufs [500MHz] → AXFSUB[1] R-channel

---

4. Interface Specifications

4.1 AXI4 Full Subordinate Interfaces — AXFSUB[0] and AXFSUB[1]

Each subordinate port accepts AXI4 Full transactions from the local SoC fabric.

Parameter	Value
Data bus width	1024 bits (128 bytes per beat)
Address width	40 bits
ID width	8 bits (AWID, ARID)
Clock	`axi_clk` — 500 MHz
Reset	`axi_rst_n` — active-low, synchronous to `axi_clk`
Burst types supported	INCR, WRAP (FIXED is accepted but mapped to INCR internally)
Max burst length	256 beats (AXI4 standard: AWLEN[7:0])
Byte enables	WSTRB[127:0] — fully supported
Exclusive access	Not supported — EXOKAY will never be returned; OKAY returned
Locked transactions	Not supported
QoS	AWQOS/ARQOS accepted; used to select VCN (see §5.3)
Region	AWREGION/ARREGION ignored
Outstanding transactions	Up to 256 per port (limited by ID space)

AXI protocol version	AXI4 Full (no AXI5 atomic extensions)

AXI4 signal list per AXFSUB port:

```
// Write Address Channel
input  [39:0]    awaddr
input  [7:0]     awid
input  [7:0]     awlen
input  [2:0]     awsize
input  [1:0]     awburst
input            awvalid
output           awready

// Write Data Channel
input  [1023:0]  wdata
input  [127:0]   wstrb
input            wlast
input            wvalid
output           wready

// Write Response Channel
output [7:0]     bid
output [1:0]     bresp
output           bvalid
input            bready

// Read Address Channel
input  [39:0]    araddr
input  [7:0]     arid
input  [7:0]     arlen
input  [2:0]     arsize
input  [1:0]     arburst
input            arvalid
output           arready

// Read Data Channel
output [1023:0]  rdata
output [7:0]     rid
output [1:0]     rresp
output           rlast
output           rvalid
input            rready
```

4.2 AXI4 Full Manager Interfaces — AXFMGR[0] and AXFMGR[1]

Each manager port drives AXI4 Full transactions into the local SoC fabric on behalf of the remote die. Signal polarity is mirrored from AXFSUB.

Parameter	Value
Data bus width	1024 bits (128 bytes per beat)
Address width	40 bits
ID width	8 bits
Clock	`axi_clk` — 500 MHz
Reset	`axi_rst_n`
Features	Same as AXFSUB, plus: tag remapping for collision avoidance

Note on ID remapping: Since both AXFMGR ports originate from different remote dies sharing a 256-ID space, the bridge shall prepend a 1-bit port discriminator to outgoing AXI IDs on shared fabric. If the fabric supports 9-bit IDs, this is transparent. Otherwise, an ID remapping table (configurable via CSR) is provided.

4.3 FDI Mainband Interface

Per UCIe 2.0 specification, the FDI mainband uses two unidirectional data paths.

Parameter	Value
Flit width	**2048 bits (256 bytes)**
Clock	`fdi_clk` — 625 MHz
Reset	`fdi_rst_n` — active-low, synchronous to `fdi_clk`
Peak bandwidth	256B × 625M = **160 GB/s** per direction

Usable bytes per flit	**248 bytes** (last 8 bytes reserved for UCIe adapter CRC — see §5.1)
Protocol	UCIe 2.0 Streaming, Flit Format 6

LP (Local Protocol → Adapter) signals:

```
output [2047:0]  fdi_lp_data        // Flit data (256 bytes)
output           fdi_lp_valid       // Flit valid
output           fdi_lp_irdy        // LP is ready to transfer (handshake)
output           fdi_lp_startofpkt  // First flit of a packet
output           fdi_lp_endofpkt    // Last flit of a packet
input            fdi_pl_trdy        // Adapter ready (backpressure)
output [2:0]     fdi_lp_state_req   // Link state request (ACTIVE/IDLE/RESET)
output           fdi_lp_linkerror   // Fatal link error
```

PL (Adapter → Local Protocol) signals:

```
input  [2047:0]  fdi_pl_data
input            fdi_pl_valid
input            fdi_pl_irdy
input            fdi_pl_startofpkt
input            fdi_pl_endofpkt
output           fdi_lp_trdy        // Die ready to accept (backpressure)
input  [2:0]     fdi_pl_state_sts   // Adapter link state status
input            fdi_pl_inband_pres // In-band presence detection
```

Handshake: A flit is transferred when `valid && irdy && trdy` are all asserted in the same cycle. The adapter may de-assert `trdy` to apply backpressure. The bridge shall not de-assert `irdy` mid-packet (stall at packet boundaries only).

4.4 APB Sideband / CSR Interface

Parameter	Value
Data width	32 bits
Address width	16 bits (64 KB CSR space)

Clock	`pclk` — asynchronous to both `axi_clk` and `fdi_clk`; assumed ≤ 200 MHz
Reset	`prst_n` — active-low
Protocol	APB4

```
input          pclk
input          prst_n
input  [15:0]   paddr
input          psel
input          penable
input          pwrite
input  [31:0]   pwdata
input  [3:0]    pstrb
output [31:0]   prdata
output         pready
output         pslverr
```

---

5. UCIe 2.0 Streaming Flit Format

5.1 Format 6 — Streaming Identification and CRC Reservation

UCIe 2.0 Streaming flits are identified by `FORMAT[2:0] = 3'b110` (decimal 6) in the flit header. The remaining header fields are application-defined.

CRC tail reservation: The UCIe D2D adapter appends an 8-byte (64-bit) CRC to each flit before transmission over the PHY. The bridge therefore drives the last 8 bytes of **every flit** (header, data, and WSTRB extension) as `0x00`. The adapter overwrites these bytes with the computed CRC. The receiver strips the CRC before presenting flits to the bridge. The bridge never reads or generates CRC values.

All flit types — byte layout:
Bytes [0 .. 247] : Usable content (header fields + payload)
Bytes [248.. 255] : CRC_RSVD — driven 0 by bridge; overwritten by UCIe adapter

Usable bytes per flit: 248 bytes (1984 bits).

All multi-bit fields are **little-endian** (LSB at lower bit index).

5.2 Header Flit Definition (256 bytes = 2048 bits total)

Every packet begins with exactly one Header Flit.

Bits	Field	Width	Description
[2:0]	FORMAT	3	Must be 3'b110 (Streaming/Format 6)
[5:3]	VCN	3	Virtual Channel Number (0–7); see §5.6
[6]	SOP	1	Start Of Packet (always 1 in header flit)
[7]	EOP	1	End Of Packet (1 = no data flits follow)
[11:8]	OPCODE	4	Transaction opcode; see §5.4
[12]	PORT_ID	1	Destination AXFMGR port (0 or 1)
[20:13]	TXN_ID	8	AXI ID (AWID or ARID)

[28:21] BURST_LEN 8 AXI AWLEN/ARLEN (beats − 1)

[30:29] BURST_TYPE 2 AXI AWBURST/ARBURST (00=FIXED,01=INCR,10=WRAP)

[33:31] BURST_SIZE 3 AXI AWSIZE/ARSIZE (log2 bytes/beat)

[35:34] RESP 2 AXI response code (WRITE_RESP/READ_RESP only)

[37:36] CREDIT_RTN_VCN 2 VCN for piggybacked credit return

[43:38] CREDIT_RTN_CNT 6 Credits returned in this flit (0 = none)

[44] WSTRB_PRESENT 1 1 = WSTRB bytes included in payload stream (see §5.5); 0 = all strobes implied 1

[84:45] ADDR 40 AXI AWADDR or ARADDR

[127:85] RSVD_A 43 Reserved, must be 0

[255:128] WSTRB_B0 128 Beat-0 write strobes (valid when WSTRB_PRESENT=1; else 0)

[1983:256] DATA_INLINE 1728 First 216 bytes of the write payload stream (see §5.5 for ordering when WSTRB_PRESENT=1)

Read-resp: first 216B of RDATA.

Zeroed for request-only flits (READ_REQ, WRITE_RESP, CREDIT_RTN, NOP).

[2047:1984] CRC_RSVD 64 Reserved for UCIe adapter CRC — bridge drives 0

Key property: A single-beat 128B AXI write fits entirely in **one header flit** (16B header fields + 16B WSTRB_B0 + 128B data = 160B ≤ 248B usable), EOP=1. No data flits needed for the common case.

5.3 Data Flit Definition (256 bytes = 2048 bits total)

Data flits follow the header flit. Each carries 248 bytes of payload plus the 8-byte CRC tail. All post-header flits use this same format regardless of whether they carry WSTRB bytes, write data, or read data — the content is determined by position in the payload stream (see §5.5).

Bits Field Width Description

—

[1983:0] DATA 1984 248 bytes of packed AXI beat payload (byte-aligned, contiguous)

[2047:1984] CRC_RSVD 64 Reserved for UCIe adapter CRC — bridge drives 0

The final data flit of a packet may be partially filled; unused bytes are driven 0. The receiver reconstructs exact byte count from `BURST_LEN` and `BURST_SIZE` in the header.

5.4 Opcode Definitions

OPCODE[3:0]	Name	Direction	Data Flits?	Description
4'h0	`WRITE_REQ`	TX→RX	Conditional	AXI write (AW+W merged). EOP=1 if entire payload fits in header inline
4'h1	`READ_REQ`	TX→RX	No	AXI read address only. EOP=1 always
4'h2	`WRITE_RESP`	RX→TX	No	AXI write response (B channel). EOP=1 always
4'h3	`READ_RESP`	RX→TX	Conditional	AXI read data (R channel). EOP=1 if payload fits inline
4'h4	`CREDIT_RTN`	Both	No	Standalone credit return. EOP=1 always
4'h5	`NOP`	Both	No	No-operation / keepalive

4'h6–F	Reserved	—	—	Must not be generated; receiver ignores

5.5 Write Payload Stream Ordering (WSTRB and Data)

There is no separate WSTRB flit type. WSTRB bytes are embedded directly in the packet's payload stream alongside write data, using the layout below. All post-header flits are standard data flits (§5.3).

When `WSTRB_PRESENT = 0` (all strobes implied all-ones):

Header inline [1983:256]: 216 bytes of write data (beats 0 onward, contiguous)

Data flits (if needed): 248 bytes each, continuing the write data stream

When `WSTRB_PRESENT = 1` (at least one beat has a non-trivial strobe):

Header flit:

[255:128] WSTRB_B0 Beat-0 strobe (16B, dedicated field)

[1983:256] Payload stream byte 0 onward:

bytes [0 .. (N−1)×16 − 1] WSTRBs for beats 1..N−1 (N−1 entries × 16B each)

bytes [(N−1)×16 .. end] Write data beats 0..N−1 (N entries × 128B each)

where N = BURST_LEN + 1

Data flits: Continue the same flat byte stream until all WSTRB + data bytes are sent

The receiver reconstructs boundaries using arithmetic on BURST_LEN and BURST_SIZE:

- WSTRB bytes in post-header stream: `BURST_LEN × 16` bytes (beats 1..N−1)
- Data bytes total: `(BURST_LEN + 1) × (1 << BURST_SIZE)` bytes

Example — 4-beat 128B write (BURST_LEN=3), WSTRB_PRESENT=1:

Header flit (248B usable):

16B header fields

16B WSTRB_B0 (beat 0)

16B WSTRB beat 1 ┐

16B WSTRB beat 2 │ payload stream bytes 0..47 (3 × 16B WSTRBs)

16B WSTRB beat 3 ┘

128B write data beat 0 ┐ payload stream bytes 48..175

20B write data beat 1 (partial, bytes 0..19)

———————————————————————

248B = exactly full (EOP=0, 1 data flit follows)

Data flit 1 (248B usable):

108B write data beat 1 remainder (bytes 20..127)

128B write data beat 2

12B write data beat 3 (bytes 0..11)

———————————————————————

248B (EOP=0, 1 more data flit)

Data flit 2 (248B usable, partially filled):

116B write data beat 3 remainder (bytes 12..127)

132B zeroed padding

———————————————————————

248B (EOP=1)

If all strobes are all-ones, `WSTRB_PRESENT=0`, WSTRB_B0 field is zeroed, and no WSTRB bytes appear in the payload stream — write data begins at payload byte 0.

5.6 Virtual Channel Assignments

VCN	Name	Traffic Type	AXI QoS Mapping
0	VC_NP	Non-Posted Requests (reads, write-addr+data)	ARQOS[3:2]=0 or AWQOS[3:2]=0
1	VC_CPL	Completions (read data, write responses)	N/A — always VC_CPL
2	VC_NP_HI	High-priority Non-Posted	ARQOS[3:2]=1 or AWQOS[3:2]=1
3	VC_CPL_HI	High-priority Completions	Tied to VC_NP_HI requests
4–7	Reserved	—	—

Credit pools are maintained separately per VCN.

6. TX Data Path — AXI → FDI

6.1 TX Request Buffers

Each AXFSUB port has an independent set of buffers:

Buffer	Depth	Width	Purpose
AW Buffer	32 entries	50b (AWID+AWLEN+AWBURST+AWSIZE+AWADDR)	Holds pending write address phase
W Buffer	64 entries	1161b (WDATA+WSTRB+WLAST)	Holds write data beats
AR Buffer	32 entries	50b	Holds pending read address phase
Completion Tracker	256 entries	10b (port_id + vcn + txn_id)	Tracks outstanding TXNs for response routing

The AW and W channels are paired before packetization: an entry is only forwarded to the TX arbiter once both the AW entry and the first W beat are available (AXI4 write data can arrive before the address; the bridge accommodates this by buffering up to 8 W beats ahead).

6.2 TX Port Merge and Packetizer Arbitration

The TX path has two distinct and independent selection stages: port merge and transaction-type arbitration inside the packetizer.

6.2.1 Port Merge — Simple Alternation

The two AXFSUB ports are merged using simple **packet-by-packet alternation**, not flit-level interleaving. Each FDI packet belongs to exactly one transaction from one port; its flits run consecutively from SOP to EOP without interleaving with the other port. Since FDI bandwidth (160 GB/s) exceeds the two-port AXI aggregate (128 GB/s), there is never bandwidth pressure between ports and no weights are needed.

Scheduling priority for each FDI flit slot, evaluated in order:

1. CREDIT_RTN pending? → send it immediately (unconditional override, §8.4)

2. Mid-packet in progress? → continue current packet's next data flit

3. Both ports have a packet ready? → alternate: Port 0 packet, then Port 1 packet, repeat

4. One port has a packet, other idle? → active port sends its packet without waiting

5. Neither port has a packet? → fdi_lp_valid = 0 (FDI idle, no flit transmitted)

No NOP flits are generated for idle cycles. `fdi_lp_valid` is simply deasserted. The FDI link resumes the next cycle a packet or credit return becomes available.

6.2.2 Transaction-Type Arbitration — WRR within Packetizer

Within the packetizer (FDI domain, 625 MHz), a WRR arbiter selects which transaction type to send next from the currently active port's queues:

Queue	WRR Weight (default)	Description
`CREDIT_RTN`	— (unconditional override)	Always pre-empts WRR; see §8.4
VC_CPL completions	2	WRITE_RESP / READ_RESP back to remote
VC_NP_HI requests	2	High-priority writes/reads (QoS[3:2]=1)
VC_NP requests	1	Normal-priority writes/reads

WRR operates by maintaining a credit counter per queue, initialised to the queue's weight at the start of each round. The highest-credit non-empty queue wins. On exhaustion all counters reset.

Example — weights [CPL=2, NP_HI=2, NP=1], all queues non-empty, no CREDIT_RTN pending:

Start of round → counters [CPL=2, NP_HI=2, NP=1]

Pick 1 → VC_CPL counters [1, 2, 1]

Pick 2 → VC_NP_HI counters [1, 1, 1]

Pick 3 → VC_CPL counters [0, 1, 1]

Pick 4 → VC_NP_HI counters [0, 0, 1]

Pick 5 → VC_NP counters [0, 0, 0] → reset to [2, 2, 1]

VC_CPL: 2/5 = 40% — VC_NP_HI: 2/5 = 40% — VC_NP: 1/5 = 20%. Normal-priority traffic is never starved; completions and high-priority requests share the bandwidth equally ahead of it.

If a queue is empty when selected the arbiter skips it without consuming its credit (credit-preserving skip). If a VCN has zero TX credits it is excluded from selection entirely for that round. WRR weights are configurable via CSR `TX_ARB_WEIGHT` (§10.1).

6.3 Packetizer

The packetizer converts the selected AXI transaction into a sequence of FDI flits:

Write transaction → WRITE_REQ packet:

1. Compose header flit: FORMAT=6, VCN, SOP=1, OPCODE=WRITE_REQ, PORT_ID, TXN_ID=AWID, BURST_LEN=AWLEN, BURST_TYPE=AWBURST, BURST_SIZE=AWSIZE, ADDR=AWADDR. Set WSTRB_PRESENT=1 if any beat has a non-trivial strobe, else 0. Embed beat-0 WSTRB at [255:128] if WSTRB_PRESENT=1. Fill DATA_INLINE [1983:256] with the first 216 bytes of the write payload stream per §5.5 ordering (remaining WSTRBs first, then write data). Drive CRC_RSVD = 0.
2. If payload stream exceeds 216B: emit data flits, each carrying 248B of the continuing payload stream (WSTRB remainder then data remainder), until all bytes are sent.
3. Set EOP=1 on the last flit — which is the header itself for single-beat 128B writes.
4. Decrement TX credit counter for VCN_NP by 1.

Common case — single-beat 128B write: Header flit only (1 flit). 16B header + 16B WSTRB_B0 + 128B data = 160B ≤ 248B usable → **EOP=1 in header, zero data flits.**

Read transaction → READ_REQ packet:

1. Compose header flit: OPCODE=READ_REQ, PORT_ID, TXN_ID=ARID, BURST_LEN/TYPE/SIZE, ADDR=ARADDR. DATA_INLINE zeroed. EOP=1 (no data flits, entire packet = 1 flit).
2. Decrement TX credit counter for VCN_NP by 1.
3. Log entry in Completion Tracker (TXN_ID, PORT_ID, VCN_CPL).

Completion forwarding → WRITE_RESP / READ_RESP: When AXFMGR[x] receives a B or R response from the local fabric, the packetizer routes it back as a WRITE_RESP or READ_RESP packet using the VCN_CPL channel. The original TXN_ID and PORT_ID were stored in the Completion Tracker at request time.

6.4 TX CDC — AXI Domain Boundary (500 MHz → 625 MHz)

CDC async FIFOs are placed **at the AXI boundary**, between the per-port input buffers (AXI domain, 500 MHz) and the WRR arbiter (FDI domain, 625 MHz). The arbiter, packetizer, credit manager TX side, and FDI driver all run at 625 MHz.

Per-port TX CDC FIFOs (2 ports × 3 FIFOs each):

FIFO	Width	Depth	Purpose
AW CDC FIFO (×2)	59b (AWID+AWLEN+AWBURST+AWSIZE+AWADDR)	16 entries	AW metadata AXI→FDI domain
W CDC FIFO (×2)	1153b (WDATA[1023:0]+WSTRB[127:0]+WLAST)	16 entries	Write data beats AXI→FDI domain
AR CDC FIFO (×2)	59b (ARID+ARLEN+ARBURST+ARSIZE+ARADDR)	16 entries	AR metadata AXI→FDI domain

Additionally, B/R responses from AXFMGR (which arrive in AXI domain) must cross back to FDI domain for packetization as WRITE_RESP/READ_RESP:

FIFO	Width	Depth	Purpose
B CDC FIFO (×2)	10b (BID+BRESP)	16 entries	Write response AXFMGR→FDI domain
R CDC FIFO (×2)	1027b (RDATA+RID+RRESP+RLAST)	16 entries	Read data AXFMGR→FDI domain

All FIFOs use gray-code pointer synchronization with 2-FF synchronizers. FIFO occupancy is monitored via CSR.

7. RX Data Path — FDI → AXI

7.1 RX CDC — AXI Domain Boundary (625 MHz → 500 MHz)

The depacketizer and port router run in the FDI domain (625 MHz). CDC async FIFOs sit at the AXI boundary, feeding the per-port RX dispatch buffers (AXI domain, 500 MHz).

Per-port RX CDC FIFOs (2 ports × 3 FIFOs each, for AXFMGR dispatch and AXFSUB response delivery):

FIFO	Width	Depth	Purpose
AW_D CDC FIFO (×2)	59b	16 entries	Inbound write address → AXFMGR AW
W_D CDC FIFO (×2)	1153b	16 entries	Inbound write data → AXFMGR W
AR_D CDC FIFO (×2)	59b	16 entries	Inbound read address → AXFMGR AR
B_D CDC FIFO (×2)	10b	16 entries	Inbound write response → AXFSUB B channel
R_D CDC FIFO (×2)	1027b	16 entries	Inbound read data → AXFSUB R channel

All FIFOs use gray-code pointer synchronization with 2-FF synchronizers.

7.2 Depacketizer

The depacketizer runs in the FDI domain (625 MHz) and parses incoming flits before the CDC boundary:

1. Detect SOP to begin a new packet.
2. Parse header flit: extract FORMAT (must be 3'b110), VCN, OPCODE, PORT_ID, TXN_ID, BURST fields, ADDR.
3. For WRITE_REQ: check WSTRB_PRESENT.
    - If WSTRB_PRESENT=1: extract WSTRB_B0 from header bits [255:128]; then from the flat payload byte stream, extract the leading `(BURST_LEN)×16` bytes as WSTRBs for beats 1..BURST_LEN; remaining bytes are write data for all beats 0..BURST_LEN.
    - If WSTRB_PRESENT=0: treat all WSTRBs as all-ones (16'hFFFF per beat); entire payload byte stream is write data.

4. Increment RX credit return counter for the received VCN (credit return signaling — see §8).
5. Route packet to the appropriate handler based on OPCODE.

7.3 Port Router and OPCODE Dispatch

OPCODE	Action
WRITE_REQ	Forward AW to AXFMGR[PORT_ID]; buffer and forward W beats; await B response; send WRITE_RESP back via TX
READ_REQ	Forward AR to AXFMGR[PORT_ID]; await R data; send READ_RESP back via TX
WRITE_RESP	Match TXN_ID to Completion Tracker; forward BRESP to AXFSUB[PORT_ID] B channel
READ_RESP	Match TXN_ID; reconstruct R channel beats; forward to AXFSUB[PORT_ID] R channel
CREDIT_RTN	Update TX credit counter for specified VCN
NOP	Discard

7.4 RX Data Buffers (per port)

Buffer	Depth	Width	Purpose
AW Dispatch Buffer	16 entries	50b	Pending writes to issue on AXFMGR
W Reassembly Buffer	64 entries	1161b	Reassembled write data beats
AR Dispatch Buffer	16 entries	50b	Pending reads to issue on AXFMGR
R Reassembly Buffer	256 entries	1025b (data+rlast)	Read data beats awaiting AXFSUB delivery

8. Credit Flow Management

8.1 Credit Model

Credit flow control prevents FDI buffer overflow at the receiver. Credits are managed **per VCN** and represent the number of packet slots available in the receiver's incoming buffer for that channel.

The bridge maintains two sets of credit counters:

- **TX Credits** — credits granted by the remote die; the local bridge consumes one credit per transmitted packet.
- **RX Credits** — credits held at the local bridge for incoming traffic; returned to the remote die as packets are consumed.

8.2 Credit Initialization

At link bring-up (after FDI reaches ACTIVE state):

1. Each side initializes its RX buffer to a fixed depth (configurable via CSR, default shown below).
2. Each side transmits initial credit grants to the remote using `CREDIT_RTN` packets (one packet per VCN with CREDIT_RTN_CNT set to the initial buffer depth).
3. The remote stores received grants in TX credit counters.
4. Normal traffic begins only after both sides have exchanged initial credits.

VCN	Default RX Buffer (= initial grant)
VC_NP	32 packets
VC_CPL	32 packets
VC_NP_HI	16 packets
VC_CPL_HI	16 packets

8.3 Credit Return Mechanism

Credits are returned to the transmitter using one of two mechanisms, chosen by the hardware dynamically:

Inline credit return: If a TX packet is ready, the credit return information is piggybacked into the packet's header flit using `CREDIT_RTN_VCN` and `CREDIT_RTN_CNT` fields. This is the preferred path as it costs zero additional bandwidth.

Standalone credit return: If no TX packet is pending but RX credits need to be returned (credit threshold exceeded), a `CREDIT_RTN` NOP-class packet is generated. This prevents transmitter starvation.

Credit return threshold: Credits are returned when the number of consumed-but-unreturned credits for any VCN reaches **half the initial grant** (configurable via CSR register `CREDIT_RTN_THRESH`).

8.4 Deadlock Avoidance

The following rules are enforced to prevent credit deadlock:

1. **No circular dependency:** Completion credits (VC_CPL) are never consumed waiting for VC_NP credits. Completions are always issued when a local AXFMGR response is received, regardless of VC_NP credit availability.
2. **Credit return priority:** Standalone credit return packets have highest TX priority, overriding the WRR arbiter.
3. **Completion buffer independence:** The R and B reassembly buffers are never shared with request buffers.
4. **Minimum TX credit hold:** The bridge shall not send a packet if TX credits = 0; it stalls the TX arbiter for that VCN.

8.5 Credit Counter CSR Visibility

All TX and RX credit counters are readable via CSR for debug purposes (see §10 CSR Map). Hardware saturates at the maximum configured value; underflow triggers an error interrupt.

---

9. Clock Domain Crossing Strategy

9.1 Clock Domains

Domain	Clock	Frequency	Reset
AXI	`axi_clk`	500 MHz	`axi_rst_n`
FDI	`fdi_clk`	625 MHz	`fdi_rst_n`
APB	`pclk`	≤ 200 MHz	`prst_n`

All three clocks are assumed to be **asynchronous** to each other (no phase relationship guaranteed).

9.2 CDC Placement and Structure

Design principle: All CDC crossings occur at the AXI interface boundary. No signals cross from AXI to FDI domain (or vice versa) anywhere inside the pipeline. The arbiter, packetizer, depacketizer, port router, and credit manager all run exclusively in the FDI domain (625 MHz).

Crossing	Direction	Structure	FIFOs / Instances	Notes
TX request path	AXI → FDI	Async FIFO (gray-code)	AW, W, AR × 2 ports = 6 FIFOs	AXI metadata and W-data to packetizer
TX completion return	AXI → FDI	Async FIFO (gray-code)	B, R × 2 ports = 4 FIFOs	AXFMGR responses back to packetizer
RX request dispatch	FDI → AXI	Async FIFO (gray-code)	AW_D, W_D, AR_D × 2 ports = 6 FIFOs	Inbound requests to AXFMGR
RX completion delivery	FDI → AXI	Async FIFO (gray-code)	B_D, R_D × 2 ports = 4 FIFOs	Responses to AXFSUB

APB → AXI (CSR writes)	pclk → AXI	2-FF synchronizer pulse	Per writable register	Register write handshake
APB → FDI (CSR writes)	pclk → FDI	2-FF synchronizer pulse	Per writable register	FDI state, credit config
AXI → APB (status)	AXI → pclk	2-FF synchronizer	Per status bit	FIFO occupancy, error flags
FDI → APB (status)	FDI → pclk	2-FF synchronizer	Per status bit	Link state, credit counters

Total async FIFOs: 20 (6 TX-req + 4 TX-cpl + 6 RX-req + 4 RX-cpl).

9.3 CDC Design Rules

- All async FIFOs use the **binary-to-gray / gray-to-binary** pointer scheme.
- Pointer synchronizers use **2-FF synchronizers** with minimum hold time constraints.
- No combinational logic is permitted across clock domains without a synchronizer.
- The wide W and R_D FIFOs (1153b and 1027b respectively) shall be implemented as synchronous RAM-based FIFOs with gray-code pointer CDC — not as shift-register FIFOs — to avoid excessive area.
- FDI reset (`fdi_rst_n`) shall be de-asserted only after `axi_rst_n` is already de-asserted (reset sequencing — see §12).

---

10. CSR Map (APB-accessible)

All registers are 32-bit, APB-accessible. Base address is implementation-defined (set during integration). Offset range: 0x0000–0x1FFF.

10.1 Global Control

Offset	Register	Access	Reset	Description
0x000	BRIDGE_CTRL	RW	0x0	[0] Bridge enable; [1] TX enable; [2] RX enable; [3] Loopback mode
0x004	BRIDGE_VERSION	RO	0x0200_0001	[31:16] Major ver; [15:8] Minor ver; [7:0] Patch
0x008	BRIDGE_STATUS	RO	0x0	[0] TX active; [1] RX active; [2] Link ACTIVE; [3] Credit init done; [7:4] FDI state
0x00C	TX_ARB_WEIGHT	RW	0x2222	[3:0] Port0-Write; [7:4] Port0-Read; [11:8] Port1-Write; [15:12] Port1-Read

10.2 Credit Configuration

Offset	Register	Access	Reset	Description
0x020	CREDIT_INIT_NP	RW	0x0020	[15:0] Initial RX credit grant for VC_NP
0x024	CREDIT_INIT_CPL	RW	0x0020	[15:0] Initial RX credit grant for VC_CPL
0x028	CREDIT_INIT_NP_HI	RW	0x0010	[15:0] Initial RX credit grant for VC_NP_HI
0x02C	CREDIT_INIT_CPL_HI	RW	0x0010	[15:0] Initial RX credit grant for VC_CPL_HI
0x030	CREDIT_RTN_THRESH	RW	0x0010	[15:0] Return threshold (same for all VCNs; configurable per-VCN in extended map)

10.3 Credit Status (Debug)

Offset	Register	Access	Description
0x040	TX_CREDIT_NP	RO	Current TX credit count for VC_NP
0x044	TX_CREDIT_CPL	RO	Current TX credit count for VC_CPL
0x048	TX_CREDIT_NP_HI	RO	Current TX credit count for VC_NP_HI
0x04C	TX_CREDIT_CPL_HI	RO	Current TX credit count for VC_CPL_HI
0x050	RX_CREDIT_CONSUMED_NP	RO	Unreturned RX credits for VC_NP
0x054	RX_CREDIT_CONSUMED_CPL	RO	Unreturned RX credits for VC_CPL

10.4 Error and Interrupt Status

Offset	Register	Access	Description
0x060	ERR_STATUS	RO/W1C	[0] TX credit underflow; [1] RX credit overflow; [2] Flit format error; [3] Opcode error; [4] AXI SLVERR received; [5] AXI DECERR received; [6] TXN timeout; [7] CDC FIFO overflow
0x064	ERR_ADDR_LO	RO	Lower 32b of address associated with last error
0x068	ERR_ADDR_HI	RO	Upper 8b of address associated with last error
0x06C	INTR_ENABLE	RW	Per-bit interrupt enable mask for ERR_STATUS
0x070	INTR_STATUS	RO/W1C	Latched interrupt status (ERR_STATUS & INTR_ENABLE)

0x074	`TXN_TIMEOUT`	RW	[15:0] AXI cycles before timeout fires (default 65535)

10.5 Performance Counters (optional, clock-gated)

Offset	Register	Description
0x100	`TX_FLIT_CNT`	Total TX flits sent
0x104	`RX_FLIT_CNT`	Total RX flits received
0x108	`TX_STALL_CYCLES`	Cycles TX was stalled (no credits)
0x10C	`RX_STALL_CYCLES`	Cycles RX was stalled (backpressure on AXI)
0x110–0x11C	`TX_PKT_CNT[0:3]`	TX packet count per VCN
0x120–0x12C	`RX_PKT_CNT[0:3]`	RX packet count per VCN

11. Error Handling

11.1 AXI Protocol Errors

AXI Error	Detection Point	Bridge Response
SLVERR on AXFMGR write	B channel BRESP[1:0]=2'b10	Packetize WRITE_RESP with RESP=SLVERR; set ERR_STATUS[4]
DECERR on AXFMGR write	B channel BRESP[1:0]=2'b11	Packetize WRITE_RESP with RESP=DECERR; set ERR_STATUS[5]
SLVERR on AXFMGR read	R channel RRESP[1:0]=2'b10	Packetize READ_RESP with RESP=SLVERR per beat
EXOKAY	Never generated by bridge	—
AXI timeout	AWVALID/ARVALID asserted >N cycles	Return SLVERR to local subordinate; set ERR_STATUS[6]

Timeout threshold N is configurable in CSR `TXN_TIMEOUT` (default: 65535 AXI clock cycles ≈ 131 µs at 500 MHz).

11.2 FDI Protocol Errors

FDI Error	Detection	Response
Invalid FORMAT field (≠ 3'b110)	Depacketizer	Discard packet; set ERR_STATUS[2]; assert `fdi_lp_linkerror`
Reserved OPCODE received	Depacketizer	Discard packet; set ERR_STATUS[3]
Credit underflow (TX sends with 0 credits)	Credit manager	Block TX; assert error interrupt
Credit overflow (RX credits exceed init grant)	Credit manager	Discard and flag ERR_STATUS[1]

CDC FIFO overflow	CDC monitor	Flag ERR_STATUS[7]; bridge enters error state

11.3 Error Recovery

The bridge supports a **soft reset** sequence triggered by writing `BRIDGE_CTRL[4]=1`. This:

1. Drains the TX FIFO (completes in-flight packets).
2. Resets all credit counters.
3. Returns to credit initialization state.
4. Re-sends initial credit grants.

A **hard reset** (asserting `axi_rst_n` or `fdi_rst_n`) flushes all buffers immediately.

---

12. Reset Architecture

12.1 Reset Domains

Reset Signal	Domain	Type	Assertion	De-assertion
`por_n`	Global	Asynchronous	Immediately	After stable power + clock
`axi_rst_n`	AXI	Synchronous to `axi_clk`	Immediate	16-cycle synchronous de-assertion
`fdi_rst_n`	FDI	Synchronous to `fdi_clk`	Immediate	16-cycle synchronous de-assertion
`prst_n`	APB	Synchronous to `pclk`	Immediate	Standard APB reset sequence

12.2 Reset Sequence

por_n de-asserts
  → axi_rst_n de-asserts (after clocks stable)
    → fdi_rst_n de-asserts (after axi domain ready)
      → prst_n de-asserts
        → Bridge asserts fdi_lp_state_req = ACTIVE
          → Credit initialization exchange
            → Normal operation

Rule: `fdi_rst_n` shall not de-assert while `axi_rst_n` is still asserted. Violation is flagged via `ERR_STATUS` if detectable in RTL.

---

13. Performance Model

13.1 Bandwidth Analysis

Parameter	Value
AXI clock	500 MHz
AXI per-port data width	1024 bits (128 bytes)
Peak per-port AXI bandwidth	128B × 500M = **64 GB/s**
Two-port aggregate AXI BW	**128 GB/s**
FDI clock	625 MHz
FDI flit width	**2048 bits (256 bytes)**
FDI usable per flit (after 8B CRC)	**248 bytes**
Peak FDI bandwidth (per direction)	256B × 625M = **160 GB/s**
FDI headroom over AXI aggregate	~25% — FDI is **not** the bottleneck
Credit flow control role	Buffer management and burst-level flow regulation; not BW-limited

13.2 Packing Efficiency

Single-beat 128B write (AWLEN=0), all-ones WSTRB:

- Header flit with 128B data inline + 16B header fields = 144B ≤ 248B → **1 flit, EOP in header**
- On-wire: 256B for 128B payload → **efficiency: 50%** (limited by single-beat, not flit size)

Single-beat 128B write, partial WSTRB:

- Header flit: 16B header fields + 16B WSTRB + 128B data = 160B ≤ 248B → still **1 flit**
- Efficiency: 50% on payload, but all info is in one flit — zero extra flits vs no-strobe case

Burst write, AWLEN=15, 16 beats × 128B = 2048B payload, all-ones WSTRB:

- Header flit: 16B header + 216B inline data = 232B (covers beat 0 + 88B of beat 1)
- Remaining: 2048 − 216 = 1832B → [1832/248] = 8 data flits (last partially filled)
- **Total: 9 flits for 2048B → payload packing efficiency: ~91%**

Max burst, AWLEN=255, 256 beats × 128B = 32768B payload:

- Header flit: 216B inline
- Remaining: 32768 − 216 = 32552B → [32552/248] = 132 data flits
- **Total: 133 flits → efficiency: ~99%**

13.3 Latency Model (Baseline, unloaded)

Event	Latency
AXI AW/W → TX buffer enqueue	1–2 AXI cycles (500 MHz)
TX CDC FIFO crossing (AXI → FDI)	~6 FDI cycles
WRR arbitration (FDI domain)	1–4 FDI cycles
Packetization — single-beat write	1 FDI cycle (1 header flit)
FDI flit transfer	1 FDI cycle
RX CDC FIFO crossing (FDI → AXI)	~6 AXI cycles
Depacketization + AXI dispatch	2–3 FDI cycles
Single-beat 128B write: TX flit latency	**~12–16 FDI cycles (~19–26 ns at 625 MHz)**
End-to-end (AXI SUB → remote AXI MGR)	~30–40 cycles total (~48–64 ns)

14. RTL / DUT Implementation Spec

14.1 Module Hierarchy

```
axfdi_bridge (top)
├── axfdi_axi_sub [0:1]       // AXI4 subordinate port handler
│   ├── axfdi_aw_buf          // Write address buffer (32-deep FIFO)
│   ├── axfdi_w_buf           // Write data buffer (64-deep FIFO)
│   └── axfdi_ar_buf          // Read address buffer (32-deep FIFO)
├── axfdi_axi_mgr [0:1]       // AXI4 manager port handler
│   ├── axfdi_aw_dispatch     // AW dispatch to local fabric
│   ├── axfdi_w_reassem       // W beat reassembly from flits
│   ├── axfdi_ar_dispatch     // AR dispatch
│   └── axfdi_r_reassem       // R beat reassembly
├── axfdi_tx_arb              // Weighted round-robin TX arbiter (4-way)
├── axfdi_packetizer          // AXI → UCIe Streaming Fmt6 flit builder
├── axfdi_tx_cdc              // Async FIFO: AXI 500MHz → FDI 625MHz
├── axfdi_depacketizer        // UCIe Streaming Fmt6 → AXI parser
├── axfdi_rx_cdc              // Async FIFO: FDI 625MHz → AXI 500MHz
├── axfdi_port_router         // OPCODE/PORT_ID dispatch to AXI manager
├── axfdi_credit_mgr          // TX/RX credit counters, return logic
│   ├── axfdi_tx_credit [0:3] // Per-VCN TX credit counter
│   └── axfdi_rx_credit [0:3] // Per-VCN RX credit + return scheduler
├── axfdi_completion_tracker  // Outstanding TXN tracking (256-entry CAM)
├── axfdi_csr                 // APB4 register block + sync bridges
│   ├── axfdi_apb_if          // APB4 slave interface
│   ├── axfdi_sync_axi        // APB→AXI domain synchronizers
│   └── axfdi_sync_fdi        // APB→FDI domain synchronizers
└── axfdi_fdi_if              // FDI LP/PL interface driver + state machine
```

14.2 Key RTL Parameters (Localparam / Parameter)

Parameter	Default	Description
`AXI_DATA_W`	1024	AXI data bus width (bits)
`AXI_ADDR_W`	40	AXI address width (bits)
`AXI_ID_W`	8	AXI ID width (bits)

`FDI_DATA_W`	256	FDI flit width (bits)
`NUM_AXI_PORTS`	2	Number of AXI sub/mgr port pairs
`NUM_VCN`	4	Number of virtual credit channels
`AW_BUF_DEPTH`	32	Write address buffer depth (per port)
`W_BUF_DEPTH`	64	Write data buffer depth (per port)
`AR_BUF_DEPTH`	32	Read address buffer depth (per port)
`TX_FIFO_DEPTH`	16	TX CDC async FIFO depth (flits)
`RX_FIFO_DEPTH`	16	RX CDC async FIFO depth (flits)
`COMPL_TRK_DEPTH`	256	Completion tracker entries
`CDC_SYNC_STAGES`	2	CDC synchronizer FF stages
`CREDIT_INIT_NP`	32	Default NP credit grant
`CREDIT_INIT_CPL`	32	Default CPL credit grant

14.3 Coding Guidelines

- RTL in **SystemVerilog** (IEEE 1800-2017). No VHDL.
- All state machines use enumerated types with explicit default states.
- No latches permitted; all combinational outputs have default assignments.
- CDC structures (async FIFOs, synchronizers) shall be in dedicated modules, clearly tagged with `// CDC_MODULE` comments for lint tools.
- All flops reset to known values on `rst_n` assertion (synchronous reset preferred; async reset only for CDC synchronizers).
- Maximum combinational depth: 8 logic levels (target 1.5 ns at 625 MHz for FDI domain).
- No use of `initial` blocks in synthesizable RTL.

- Parametrized FIFOs from a vetted internal library (or provide synthesizable implementation within the project).
- All buses follow little-endian byte ordering (LSB = byte 0).

14.4 Clock/Reset Port Naming Convention

```
axi_clk        // AXI domain clock, 500 MHz
axi_rst_n      // AXI domain synchronous active-low reset
fdi_clk        // FDI domain clock, 625 MHz
fdi_rst_n      // FDI domain synchronous active-low reset
pclk           // APB domain clock, ≤200 MHz
prst_n         // APB domain active-low reset
```

14.5 Lint and CDC Checks

Check Category	Tool Target	Notes
Structural lint	SpyGlass / JasperGold	No undriven nets, no floating inputs
CDC analysis	VC SpyGlass CDC or Questa CDC	All CDC paths must be waived or verified
Reset domain crossing	SpyGlass CDC	Must flag any reset used outside its domain
Naming conventions	Custom lint rule	Enforce `_n` suffix for active-low, `_clk` for clocks
Simulation lint	VCS/Questa -sv warnings	No implicit net declarations

14.6 Functional Coverage Points (RTL perspective)

The following corner cases must be covered by directed or constrained-random tests:

- Simultaneous AWVALID on both AXFSUB ports
- W beats arriving before AW (write data ahead of address)
- Maximum burst length (AWLEN=255) on both ports simultaneously
- WSTRB with partial byte enables across multi-beat bursts (WSTRB_PRESENT=1, WSTRBs embedded in payload stream)
- FDI backpressure (fdi_pl_trdy deasserted) mid-packet
- TX credit exhaustion on all VCNs simultaneously

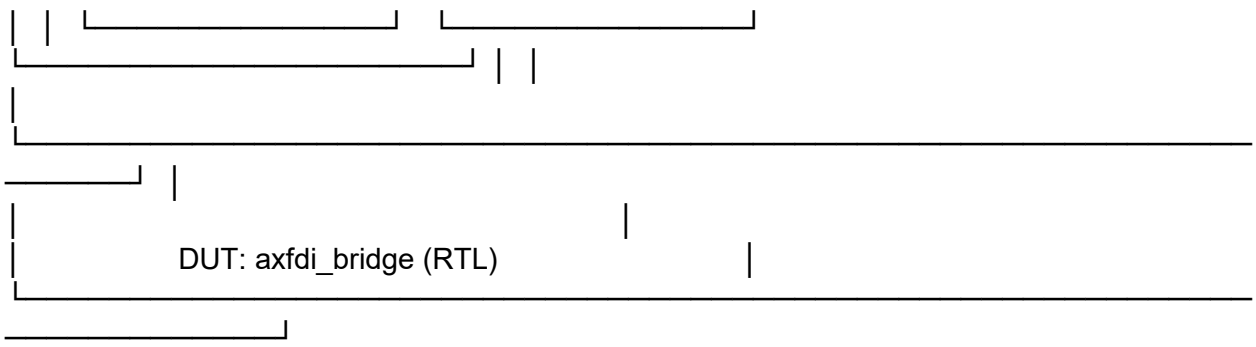
- Credit return via inline piggyback vs standalone CREDIT_RTN
- Credit initialization after reset
- Soft reset during active traffic
- CDC FIFO near-full condition
- AXI SLVERR and DECERR propagation end-to-end
- Transaction timeout
- Simultaneous reads and writes on same port
- Out-of-order read completions (different IDs)

---

15. UVM Testbench Spec

15.1 TB Architecture Overview





15.2 VIP Definitions

15.2.1 VIP-1: AXI4 Full Agent (`axi4_agent`)

One instance per port pair (one agent handles both the SUB and MGR side of the same port, sharing a sequence library but using separate drivers/monitors).

Components:

- `axi4_driver` — drives AXI4 subordinate stimulus (AW, W, AR channels). Configurable as MASTER (drives) or SLAVE (responds).
- `axi4_responder` — responds to AXFMGR transactions (simulates downstream SoC slave on the manager side). Generates B and R responses with configurable delay and error injection.
- `axi4_monitor` — passive monitor for all 5 AXI channels; outputs `axi4_seq_item` to analysis ports.
- `axi4_sequencer` — standard UVM sequencer.
- `axi4_coverage` — covergroup for protocol compliance and functional coverage.

Key sequence items (`axi4_seq_item`):

```
class axi4_seq_item extends uvm_sequence_item;
  rand bit [39:0]    addr;
  rand bit [7:0]     id;
  rand bit [7:0]     len;       // AWLEN/ARLEN
  rand bit [2:0]     size;      // AWSIZE/ARSIZE
  rand bit [1:0]     burst;     // AWBURST/ARBURST
  rand bit [1023:0]  data[];    // write/read data array
  rand bit [127:0]   strb[];    // write strobes per beat
  rand bit [1:0]     resp;      // expected/actual BRESP/RRESP
  rand bit [3:0]     qos;       // AWQOS/ARQOS
  rand axn_type_e    axn_type;  // READ / WRITE
  rand bit [31:0]    delay_cycles; // inter-transaction delay
```

endclass

Constraints classes provided:

- `axi4_full_burst_c` — valid burst: size × (len+1) ≤ 4KB boundary
- `axi4_aligned_c` — address aligned to AWSIZE
- `axi4_max_outstanding_c` — max 256 outstanding per port
- `axi4_error_inject_c` — random SLVERR/DECERR responses from responder

15.2.2 VIP-2: FDI Mainband Agent (`fdi_agent`)

Represents the UCIe Adapter side of the FDI interface. Acts as both a traffic sink (receiving LP flits) and traffic source (generating PL flits = simulated remote die responses).

Components:

- `fdi_lp_monitor` — monitors fdi_lp_* signals; decodes Format 6 packets; outputs `fdi_pkt_item`.
- `fdi_pl_driver` — drives fdi_pl_* signals; encodes `fdi_pkt_item` objects as Format 6 flits; models adapter backpressure.
- `fdi_credit_model` — software model of credit state on the adapter side; validates credit protocol compliance.
- `fdi_sequencer` — UVM sequencer for PL-side stimulus (simulating remote die traffic).
- `fdi_coverage` — covergroup for flit format, opcode, VCN distribution, credit utilization.

Key sequence items (`fdi_pkt_item`):

```
class fdi_pkt_item extends uvm_sequence_item;
  rand bit [2:0]    vcn;
  rand bit [3:0]    opcode;
  rand bit          port_id;
  rand bit [7:0]    txn_id;
  rand bit [7:0]    burst_len;
  rand bit [1:0]    burst_type;
  rand bit [2:0]    burst_size;
  rand bit [1:0]    resp;
  rand bit [39:0]   addr;
  rand bit [7:0]    data[];     // byte array payload
  rand bit [127:0]  wstrb[];    // write strobes
  rand int          backpressure_cycles; // PL trdy deassert duration
  rand bit [5:0]    credit_rtn_cnt;
  rand bit [1:0]    credit_rtn_vcn;
```

endclass

Sequences provided:

- `fdi_credit_init_seq` — sends initial credit grants at link bring-up
- `fdi_bp_seq` — applies random backpressure patterns (trdy deassert)
- `fdi_credit_starve_seq` — withholds credits to stress-test stall behavior
- `fdi_rnd_response_seq` — generates random read/write completions for AXFMGR-originated transactions

15.2.3 VIP-3: APB Agent (`apb_agent`)

Lightweight APB4 master VIP for CSR programming.

Components:

- `apb_driver` — drives psel, penable, pwrite, paddr, pwdata, pstrb; handles pready wait states.
- `apb_monitor` — captures all APB transactions to analysis port.
- `apb_sequencer` — UVM sequencer.

Key sequences:

- `csr_write_seq` — single register write
- `csr_read_seq` — single register read + expected value check
- `csr_reg_model_seq` — uses RAL (Register Abstraction Layer) model for register-level verification
- `credit_config_seq` — configures credit pool sizes and thresholds

RAL Model: A UVM RAL model (`axfdi_reg_block`) shall be generated for the CSR map in §10. All register accesses in tests should go through the RAL model to enable frontdoor/backdoor access and coverage.

15.3 Scoreboard (`axfdi_scoreboard`)

The scoreboard performs end-to-end transaction checking. It has analysis FIFOs connected to all three VIPs.

Checking algorithm:

1. When a write/read request is observed on AXFSUB[x] by the AXI monitor → log expected transaction (addr, id, data, strobe).

2. When the FDI LP monitor observes the corresponding WRITE_REQ/READ_REQ flit → verify packet contents match AXI request (address, length, opcode, port_id, data packing).
3. When the FDI PL driver sends READ_RESP/WRITE_RESP → log expected completion.
4. When AXI monitor observes B/R response on AXFSUB[x] → match to expected completion; verify RESP field, data content, and ID.
5. Cross-check: every request must have exactly one corresponding completion (no duplicates, no drops).

Credit checker: Independently tracks credit grant/consume/return state; flags any violation of credit rules.

Timeout checker: Flags any request not completed within configurable window (default: 50,000 ns).

15.4 Protocol Checkers (SVA)

SystemVerilog Assertions bound to the DUT interfaces:

AXI4 Subordinate Assertions (per port):

```
// AWVALID must not deassert without AWREADY handshake
property axi_aw_stable;
  @(posedge axi_clk) disable iff (!axi_rst_n)
  awvalid && !awready |=> awvalid && $stable(awid) && $stable(awaddr);
endproperty

// WLAST must match AWLEN
// BID must match AWID of outstanding write
// No interleaved write data (AXI4 rule)
// RVALID must not assert without prior ARVALID-ARREADY
```

FDI Mainband Assertions:

```
// FORMAT field must be 3'b110 on any valid flit
property fdi_format_check;
  @(posedge fdi_clk) disable iff (!fdi_rst_n)
  fdi_lp_valid && fdi_lp_startofpkt |-> fdi_lp_data[2:0] == 3'b110;
endproperty

// SOP must be asserted exactly on first flit of packet
// EOP must be asserted exactly on last flit
// irdy must not deassert mid-packet
// Credit counter must never go negative (shadow counter in SVA)
```

APB Assertions:

// penable must follow psel after exactly 1 cycle
// pready must deassert after transfer
// pslverr only valid when pready is asserted

15.5 Test Plan Summary

Test	Type	Priority	Coverage Goal
`t_reset_seq`	Directed	P0	Reset/bring-up; credit init
`t_single_write`	Directed	P0	Write path end-to-end, both ports
`t_single_read`	Directed	P0	Read path end-to-end, both ports
`t_burst_write_max`	Directed	P0	AWLEN=255 write on both ports
`t_burst_read_max`	Directed	P0	ARLEN=255 read on both ports
`t_rw_concurrent`	Directed	P0	Simultaneous read+write on same port
`t_dual_port_concurrent`	Directed	P0	Both ports active simultaneously
`t_wstrb_partial`	Directed	P1	WSTRB extension flit path
`t_qos_hi_priority`	Directed	P1	VC_NP_HI selection via QoS
`t_credit_starve`	Directed	P1	TX stalls when credits = 0
`t_credit_return_inline`	Directed	P1	Piggyback credit return in header

t_credit_return_standalone	Directed	P1	Standalone CREDIT_RTN packet
t_credit_init	Directed	P0	Full credit initialization sequence
t_fdi_backpressure	Directed	P1	FDI trdy deassert mid-packet
t_axi_slverr_prop	Directed	P1	SLVERR propagation from remote
t_axi_decerr_prop	Directed	P1	DECERR propagation from remote
t_timeout	Directed	P1	Transaction timeout → SLVERR
t_soft_reset	Directed	P1	Soft reset during active traffic
t_csr_rw	Directed	P0	All CSR registers read/write via APB
t_loopback	Directed	P2	Internal loopback mode
t_rand_traffic	Constrained-random	P0	General random with scoreboard
t_rand_burst_mix	Constrained-random	P1	Mixed burst types/lengths
t_rand_id_ooorder	Constrained-random	P1	OoO completions, multiple IDs
t_rand_credit_stress	Constrained-random	P1	Random credit return timing
t_rand_backpressure	Constrained-random	P1	Random FDI + AXI backpressure

`t_rand_error_inject`	Constrained-random	P2	Random SLVERR/DECERR injection
`t_cdc_stress`	Constrained-random	P2	Near-simultaneous CDC FIFO full/empty
`t_regression_full`	Random	P0	Full regression with all features

15.6 Coverage Model

Functional Coverage Groups:

- `cg_axi_burst`: cross(burst_type, burst_len[0/1/max/rand], burst_size[0:7])
- `cg_axi_id_depth`: outstanding transaction count per port (0, 1, 8, 64, 256)
- `cg_axi_wstrb`: all-ones, partial, all-zeros per port
- `cg_axi_concurrent`: both ports active vs one idle
- `cg_fdi_opcode`: all opcodes generated and received
- `cg_fdi_vcn`: all active VCNs used (0,1,2,3)
- `cg_credit_level`: TX credits at 0, 1, half, full per VCN
- `cg_credit_return`: inline vs standalone return per VCN
- `cg_cdc_fifo`: TX and RX FIFO at >75% full
- `cg_error_types`: all ERR_STATUS bits asserted
- `cg_csr_access`: all registers written and read via APB
- `cg_reset_scenario`: por, axi_rst, fdi_rst, soft_rst during active traffic

Target coverage: 95% functional coverage before sign-off.

15.7 Simulation Environment

Tool	Usage
Simulator	Synopsys VCS or Cadence Xcelium (team choice)
UVM version	UVM 1.2
Waveform	FSDB (Verdi) or SHME (Xcelium)
Coverage merge	URG (VCS) or Cadence vManager

Regression	Python-based run script; JSON-defined test list
Linting	SpyGlass (structural) + Questa Lint
Formal (optional)	JasperGold for SVA proof of credit deadlock freedom

16. Open Items / Assumptions Log

ID	Item	Decision / Assumption	Status
OI-01	AXI protocol flavor	AXI4 Full	✅ Resolved
OI-02	AXI port roles	Bidirectional symmetric (AXFSUB + AXFMGR)	✅ Resolved
OI-03	FDI flit width	256 bits at 625 MHz	✅ Resolved
OI-04	FDI protocol	UCIe 2.0 Streaming, Format 6	✅ Resolved
OI-05	AXI address width	40 bits	✅ Resolved
OI-06	AXI ID width	8 bits per port	✅ Resolved
OI-07	VIP strategy	Lightweight custom UVM VIPs	✅ Resolved
OI-08	Exclusive access	Not supported (EXOKAY never returned)	**Confirm with SoC team**

OI-09	FIXED burst support	Accepted, treated as INCR internally	**Confirm with SoC team**
OI-10	AXI ID remapping on AXFMGR	1-bit port prefix vs CSR-based table	**Open — choose strategy**
OI-11	Timeout value	65535 AXI cycles (~131 µs) default	**Confirm with system team**
OI-12	APB clock frequency	≤ 200 MHz, asynchronous to axi/fdi	**Confirm at integration**
OI-13	Power management / PM states	Not in scope for v1.0	**Confirm for v1.0**
OI-14	WSTRB multi-beat packing	WSTRB Extension flit per 2 beats	**Review packing scheme**
OI-15	Completion ordering	OoO per ID within a port (AXI4 rules)	**Confirm — no cross-ID ordering**
OI-16	UCIe Adapter retransmission	Handled below FDI; bridge assumes lossless	**Confirm with UCIe Adapter team**
OI-17	FDI link PM states	ACTIVE/IDLE/RESET only; other PM deferred	**Confirm for v1.0**

OI-18	Formal verification scope	Optional JasperGold for credit deadlock	**Team decision**
OI-19	UVM simulator choice	VCS or Xcelium — team to decide	**Open**
OI-20	WSTRB_PRESENT=0 optimization	When all strobes are all-ones, omit WSTRB bytes from payload stream entirely	**Confirmed in §5.5**

End of Specification v0.1 — AXFDI_BRIDGE Document is ready for team review. Open items (§16) require resolution before RTL kick-off.

Note 1

Port-to-port merge — simple structural merge works

Since each AXI port is 128B wide and the FDI flit data payload is 248B (usable), the bandwidth math doesn't actually give a clean "128B × 2 = 256B flit" packing for full packets because each packet has its own header. But for the data phase of simultaneous write transactions, you're right that the two ports together naturally fill the FDI flit width — no WRR needed, just a structural interleave. When only one port is active, the other half of the flit is padded. This is simpler and avoids unnecessary starvation logic between ports.

WRR — where it actually applies

WRR makes sense when you have multiple *logical channels competing for the same resource with unequal priority*, not when the bandwidth adds up cleanly. The right place is within the packetizer's selection of which transaction type (write, read, completion, credit return) to send next, and which VCN to prioritize.

Here's a concrete example. Say the packetizer has these four pending queues with weights [4, 2, 1, 1]:

Queue	Weight	Pending	
VC_NP_HI req	4	yes	
VC_NP req	2	yes	
VC_CPL	1	yes	
CREDIT_RTN	1	yes	← highest actual priority (deadlock prevention)

WRR maintains a per-queue credit counter. Each round:

Cycle	Queue selected	Remaining credits [NP_HI, NP, CPL, CRTN]	
1	CREDIT_RTN	[4, 2, 1, 0]	← always forced first (override)
2	VC_NP_HI	[3, 2, 1, —]	
3	VC_NP_HI	[2, 2, 1, —]	
4	VC_NP	[2, 1, 1, —]	
5	VC_NP_HI	[1, 1, 1, —]	

6 VC_CPL [1, 1, 0, —]

7 VC_NP_HI [0, 1, —, —]

8 VC_NP [0, 0, —, —] → reset all to [4, 2, 1, 1]

VC_NP_HI gets 4 out of 8 slots = 50% of FDI bandwidth, VC_NP gets 25%, CPL gets 12.5% — but note CREDIT_RTN always jumps the queue entirely regardless of WRR state (deadlock rule from §8.4).

So the spec needs updating:

The current spec applies WRR across all four inputs (Port0-Write, Port0-Read, Port1-Write, Port1-Read) which conflates port selection with channel-type selection. The cleaner architecture based on your observation is:

- **Port merge**: structural, simultaneous, no arbitration
- **Transaction-type/VCN selection**: WRR within the packetizer across write-req, read-req, completion, credit-return queues

The WRR doesn't see 5 independent channel FIFOs. It operates over **4 transaction-type queues**, and the AXI channels map into those as follows:

- **AW + W are always paired** before they ever reach the WRR. A write entry only enters the WRR queue once the AW metadata and the first W beat are both available. The WRR treats the whole write — address + all data beats — as a single atomic unit. It never independently picks "serve AW now, serve W later."
- **AR** maps to one queue entry (read request — just the address, no data).
- **B** (write response coming back from AXFMGR) maps to one completion queue entry.
- **R** (read data coming back from AXFMGR) maps to one completion queue entry.

So the four WRR inputs are:

WRR input Fed by AXI channel(s)

———————————————————————————————

Write request AW + W (paired, gated on both available)

Read request AR

Write completion B (from AXFMGR, returning to remote)

Read completion R (from AXFMGR, returning to remote)

The 5 AXI channel FIFOs (AW, W, AR, B, R) are the buffering layer sitting before this — they absorb the AXI handshake and hold data until the packetizer is ready. The WRR never touches individual channel FIFOs directly; it only sees whole transactions assembled from them.

The **packetizer** is the entity reading from the FIFOs, but it doesn't make an independent decision. The decision chain is:

WRR output (transaction type)

↓

Packetizer state machine knows exactly which FIFO(s) to drain

↓

Reads from those FIFOs to fill the 128B slot

Once WRR has selected a transaction type for the active port, the FIFO routing is fully deterministic:

WRR selects →	Packetizer reads from
Write request	AW FIFO (once, for header fields)
	W FIFO (repeatedly, 128B/beat, for data payload)
Read request	AR FIFO (once, header only — no data slot needed)
Write completion	B FIFO (once, header only — BRESP, no data slot)
Read completion	R FIFO (repeatedly, 128B/beat, for data payload)

So for filling that 128B data slot: it is always either the **W FIFO** (for an outbound write) or the **R FIFO** (for a read completion being returned). The packetizer's internal state machine tracks which beat it is currently on within a multi-beat transaction and keeps popping from the same FIFO until WLAST or RLAST is seen, at which point it declares the transaction done and hands control back to the WRR for the next selection.

The WRR never looks at FIFOs directly — it only looks at **non-empty flags** per transaction-type queue to decide what to pick next. The actual FIFO read pointers are owned entirely by the packetizer.

The split is **not as clean as the merge**, and for a fundamental reason: on the merge side you control the output pattern (alternating ports, WRR for types). On the split side the remote die controls what arrives and in what order. You just react.

How the split works

The depacketizer (FDI domain, 625 MHz) reads each incoming flit, extracts the PORT_ID and OPCODE from the header, reassembles the byte stream back into 128B AXI beats (inverse of the packing done on TX), and writes into the appropriate RX CDC FIFO. A packet destined for Port 0 goes into Port 0's FIFOs; Port 1 into Port 1's. The routing is clean — just a header field decode. The messiness is in what comes next.

Draining the RX FIFOs to the AXI side

Each RX CDC FIFO is drained independently by its AXI domain handler at 500 MHz:

- AW_D FIFO drains when the entry is at the head AND `awready` is asserted by the local fabric
- W_D FIFO drains one 128B beat at a time gated by `wready`
- AR_D FIFO drains when `arready` is asserted
- R_D FIFO drains one beat at a time gated by `rready`
- B_D FIFO drains gated by `bready`

Each port is fully independent — AXFMGR[0] being stalled by its downstream fabric has no effect on AXFMGR[1].

Where it gets asymmetric

On the merge you guaranteed balanced port loading by alternating. On the split the remote die might send 20 consecutive packets all to Port 0 and nothing to Port 1. Port 0's FIFOs fill up while Port 1's sit empty. If the local fabric behind AXFMGR[0] is also slow (`awready` toggling rarely), the W_D and AW_D FIFOs for Port 0 fill toward their depth limit.

Credits handle the overflow prevention

This is exactly why the RX credit system exists on this side. The credits we granted to the remote at link bring-up represent the depth of our RX FIFOs. The remote cannot send more packets to Port 0 than we have RX buffer space for. So the FIFO never overflows regardless of how lopsided or bursty the incoming traffic is.

Credits are returned as the AXI side drains the FIFOs — when `wready` or `rready` fires and a beat is consumed, the consumed-credit counter increments, and once it crosses the return threshold the credit manager generates a return (inline piggybacked on the next TX packet, or a standalone CREDIT_RTN). So the loop is:

Remote sends packet → RX FIFO fills (credit consumed)
↓
AXI fabric accepts beat → FIFO drains
↓
Consumed-credit counter hits threshold → credit returned to remote
↓
Remote can send again

The backpressure therefore never touches the FDI link directly — it stays inside the credit accounting. The FDI itself keeps flowing; the remote just self-throttles when its TX credit count hits zero. That is the key design benefit of doing flow control with credits rather than propagating `trdy` back across the link.